Supply Chain Demand Forecasting Analysis

GURJOT SINGH BAJAJ 22124040

1 Introduction

This project focuses on developing a comprehensive supply chain demand forecasting system using advanced time series analysis techniques. The primary goal is to analyze and predict product demand patterns across different categories while incorporating seasonal trends and market variations. The analysis aims to provide actionable insights for inventory management and supply chain optimization.

1.1 Dataset

The analysis utilizes the Supermart Grocery Sales Retail Analytics Dataset from Kaggle, which contains detailed sales data including:

- Order details (ID, date)
- Product categories and subcategories
- Regional information
- Sales metrics (amount, discount, profit)
- Temporal coverage: 2015-2018
- Multiple regions across Tamil Nadu, India

2 Methodology

2.1 Tools and Technologies

The project employs several Python libraries and statistical techniques:

- **Pandas**: For data manipulation and time series handling
- **Statsmodels**: For implementing SARIMA (Seasonal ARIMA) models
- **Scikit-learn**: For model evaluation metrics
- **Matplotlib & Seaborn**: For data visualization

2.2 Analysis Components

2.2.1 1. Seasonal Analysis

- Definition of seasons (based on Indian climate):
  - Winter: December, January, February
  - Summer: March, April, May

  - Monsoon: June, July, August, September
  - Post-Monsoon: October, November
- Calculation of seasonal sales distribution
- Analysis of category-wise seasonal performance
- Identification of peak and low seasons for each category

2.2.2 2. Time Series Analysis

- Monthly sales aggregation to reduce daily noise
- Year-over-year comparison for trend identification
- Seasonal pattern identification and quantification
- Trend analysis for long-term growth/decline

2.2.3 3. Demand Forecasting

- SARIMA modeling with parameters (1,1,1) for non-seasonal and (1,1,1,12) for seasonal components
- Train-test split (80-20) for model validation
- Performance evaluation using MSE and MAE metrics
- Forward forecasting for future demand prediction

3 Code Implementation

This section presents key code snippets that demonstrate the implementation of our analysis pipeline.

3.1 Data Preprocessing and Seasonal Analysis

The following code shows how we process the data and perform seasonal analysis:

```
# Function to determine season based on month
def get_season(date):
    month = date.month
    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Summer'
    elif month in [6, 7, 8, 9]:
        return 'Monsoon'
    else:
        return 'Post-Monsoon'

# Add season column
df['Season'] = df['Order Date'].apply(get_season)

# Calculate seasonal metrics
seasonal_analysis = data.groupby(['Category', 'Season'])['Sales'].agg([
    ('Total Sales', 'sum'),
    ('Average Daily Sales', 'mean'),
```

```
    ('Number of Orders', 'count')
]).reset_index()
```

Listing 1: Data Preprocessing and Seasonal Analysis

3.2 Time Series Analysis

The time series analysis implementation includes monthly aggregation and visualization:

```
def analyze_time_series(data, category):
    # Aggregate data by month
    monthly_data = data[data['Category'] == category].set_index('Order
        Date')
    monthly_data = monthly_data['Sales'].resample('M').sum()

    # Create the plot
    fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(15, 12))

    # Plot 1: Monthly Sales
    ax1.plot(monthly_data.index, monthly_data.values, marker='o')
    ax1.set_title(f'Monthly Sales - {category}')

    # Plot 2: Year-over-Year Comparison
    years = monthly_data.index.year.unique()
    for year in years:
        year_data = monthly_data[monthly_data.index.year == year]
        ax2.plot(range(1, 13), year_data.values[:12],
                 marker='o', label=str(year))

    # Plot 3: Seasonal Pattern
    seasonal_avg = monthly_data.groupby(monthly_data.index.month).mean
        ()
    ax3.plot(range(1, 13), seasonal_avg.values, marker='o', color='
        green')
```

Listing 2: Time Series Analysis Implementation

3.3 Forecasting Model

The SARIMA model implementation for demand forecasting:

```
def build_sarima_model(data, category):
    # Aggregate data by month
    monthly_data = data[data['Category'] == category].set_index('Order
        Date')
    monthly_data = monthly_data['Sales'].resample('M').sum()

    # Split data into train and test sets
    train_size = int(len(monthly_data) * 0.8)
    train_data = monthly_data[:train_size]
    test_data = monthly_data[train_size:]

    # Build SARIMA model
    model = SARIMAX(train_data,
                    order=(1, 1, 1),
                    seasonal_order=(1, 1, 1, 12))
    results = model.fit()

```

```
    # Make predictions
    forecast = results.forecast(steps=len(test_data))

    # Calculate metrics
    mse = mean_squared_error(test_data, forecast)
    mae = mean_absolute_error(test_data, forecast)
```

Listing 3: SARIMA Model Implementation

3.4 Visualization and Output Management

The code for managing visualizations and saving outputs:

```
def save_plot(fig, filename):
    """Save plot to output directory with consistent styling"""
    output_path = os.path.join('output', filename)
    fig.tight_layout()
    fig.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close(fig)

# Create output directory if it doesn't exist
os.makedirs('output', exist_ok=True)

# Set style for better visualizations
plt.style.use('seaborn')
sns.set_palette("husl")
```

Listing 4: Visualization and Output Management

4 Implementation Process

When executing demand_forecasting.py, the following steps are performed:

1. **Data Preprocessing**
   - Loading and cleaning the dataset
   - Converting dates to datetime format
   - Adding seasonal information
   - Handling missing values and outliers
2. **Seasonal Analysis**
   - Calculating sales distribution across seasons
   - Generating heatmap visualization
   - Creating average sales comparison
   - Identifying category-specific seasonal patterns
3. **Time Series Analysis**
   - Aggregating data to monthly level
   - Creating three-panel visualization:
     - Monthly sales trend
     - Year-over-year comparison

  - Average seasonal pattern
- Analyzing trend components

4. **Forecasting**
   - Building SARIMA models for each category
   - Generating forecasts
   - Calculating error metrics
   - Validating predictions

5 Results

5.1 Seasonal Patterns

The analysis revealed strong seasonal patterns across all product categories:

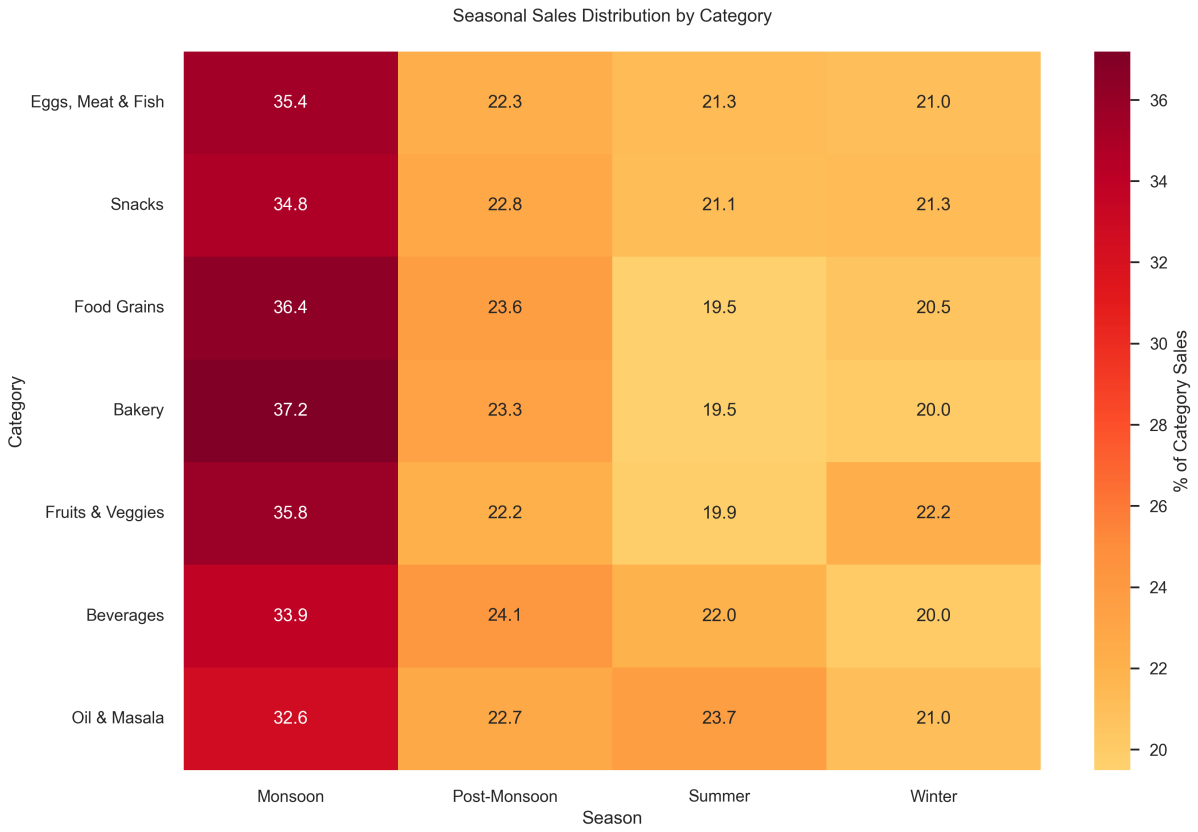

Figure 1: Seasonal Sales Distribution Heatmap - This visualization shows the percentage of total sales for each category across different seasons. Darker colors indicate higher sales percentages. Note the consistent high performance during the Monsoon season across all categories.

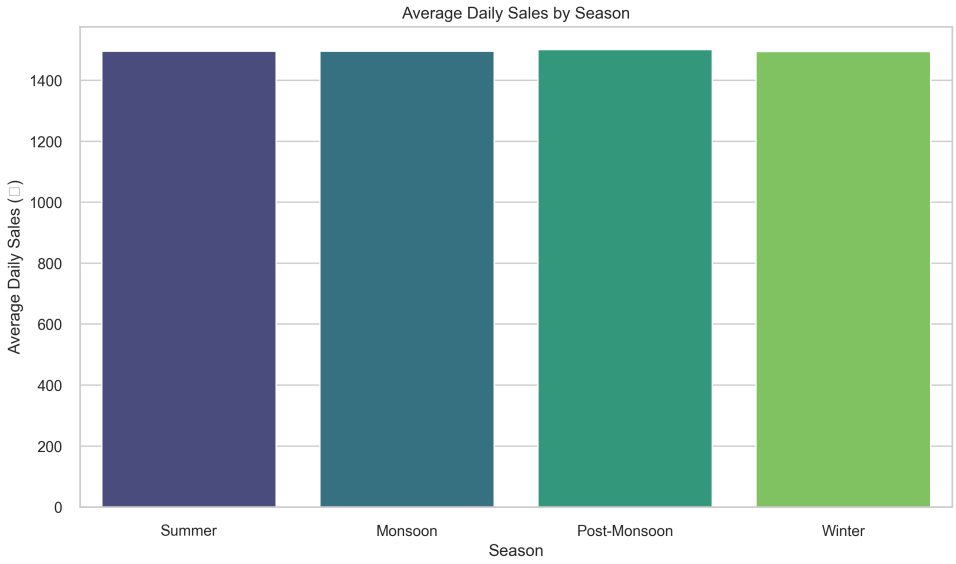

Figure 2: Average Sales by Season - Bar chart showing the overall sales performance across seasons. The Monsoon season shows significantly higher average sales, while Summer shows the lowest performance.

- **Monsoon Season Dominance**:
  - Bakery: 37.19% of annual sales (highest seasonal concentration)
  - Food Grains: 36.41% (essential items showing strong seasonal pattern)
  - Fruits & Veggies: 35.75% (fresh produce peak)
  - Eggs, Meat & Fish: 35.37% (protein category seasonal peak)
- **Seasonal Lows**:
  - Summer: Most categories show lowest sales (19-21%), possibly due to storage concerns
  - Winter: Some categories (Oil & Masala, Beverages) show reduced performance

5.2 Time Series Analysis

A detailed time series analysis was performed for each category. We present analyses for key categories:

Figure 3: Time Series Analysis - Food Grains. Top panel shows overall trend, middle panel compares yearly patterns, and bottom panel shows average monthly pattern. Note the consistent seasonal peaks and increasing trend over years.

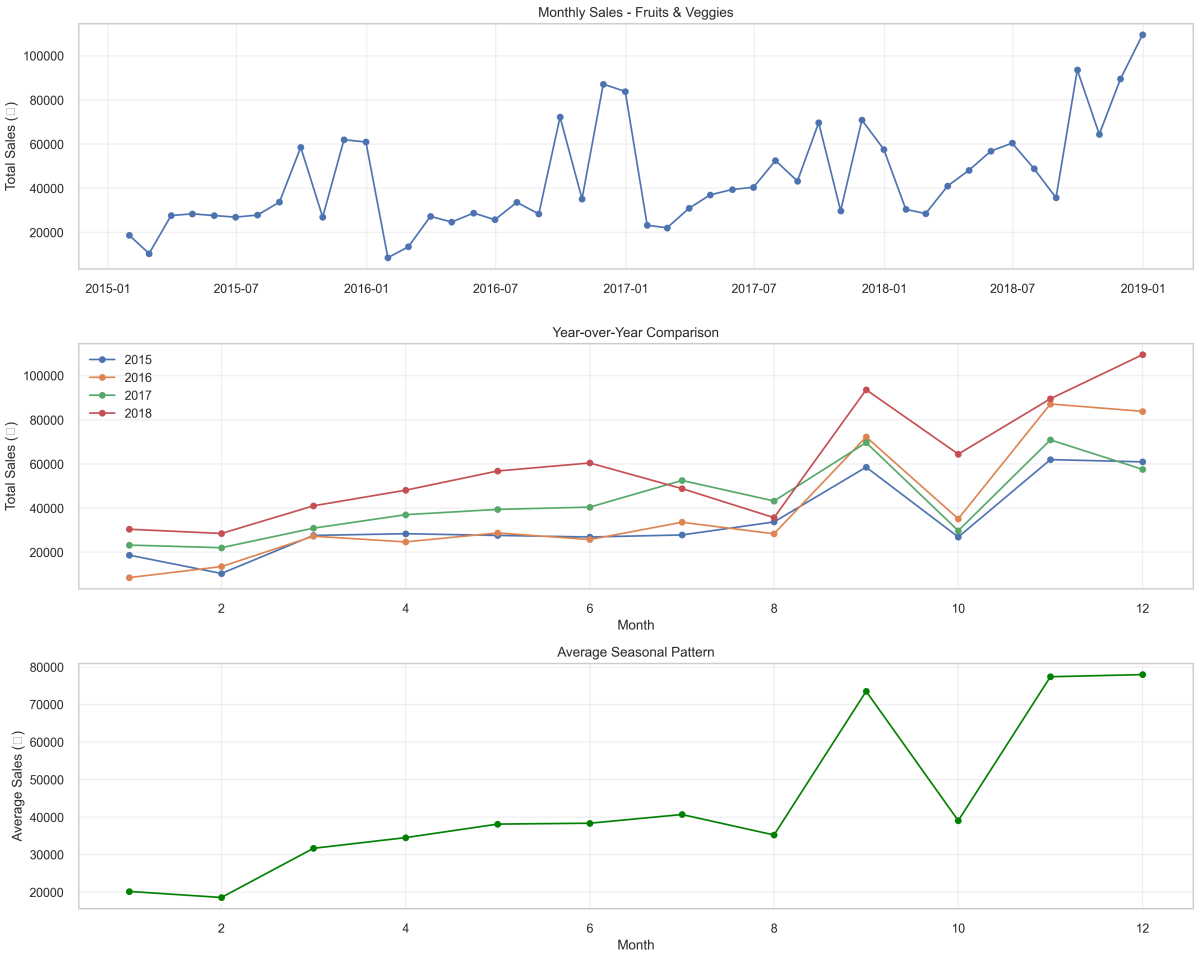

Figure 4: Time Series Analysis - Fruits & Veggies. Shows higher volatility compared to Food Grains but maintains clear seasonal patterns. Note the pronounced peaks during monsoon months.

The visualizations reveal:

- Clear monthly sales trends with consistent seasonal patterns
- Year-over-year comparison showing growth in most categories
- Distinct seasonal patterns in the average monthly sales
- Different volatility levels across categories

5.3 Forecasting Performance

Model performance varies significantly across categories:

Category	MSE	MAE
Snacks	146,785,086	10,539
Oil & Masala	156,291,153	10,248
Beverages	219,008,593	13,651
Food Grains	225,223,899	12,066
Fruits & Veggies	477,083,430	16,774

Table 1: Model Performance Metrics - Lower MSE and MAE indicate better forecast accuracy

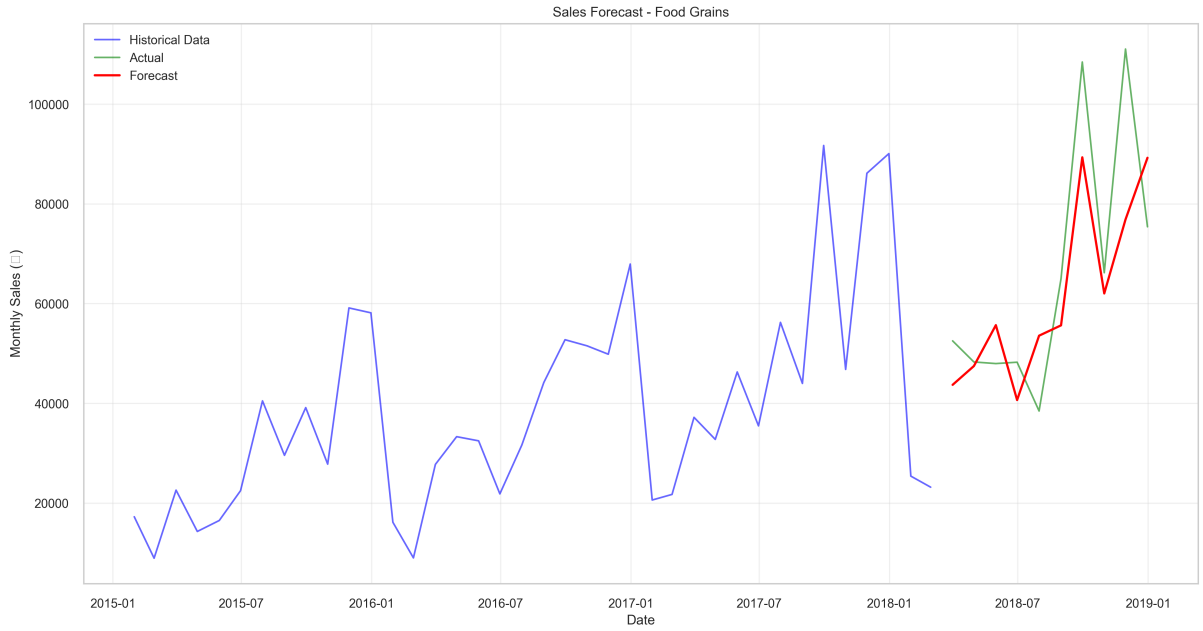

Figure 5: Sales Forecast - Food Grains. Blue shows historical data, green shows actual test data, and red shows model predictions. Note the model's ability to capture both trend and seasonal patterns.

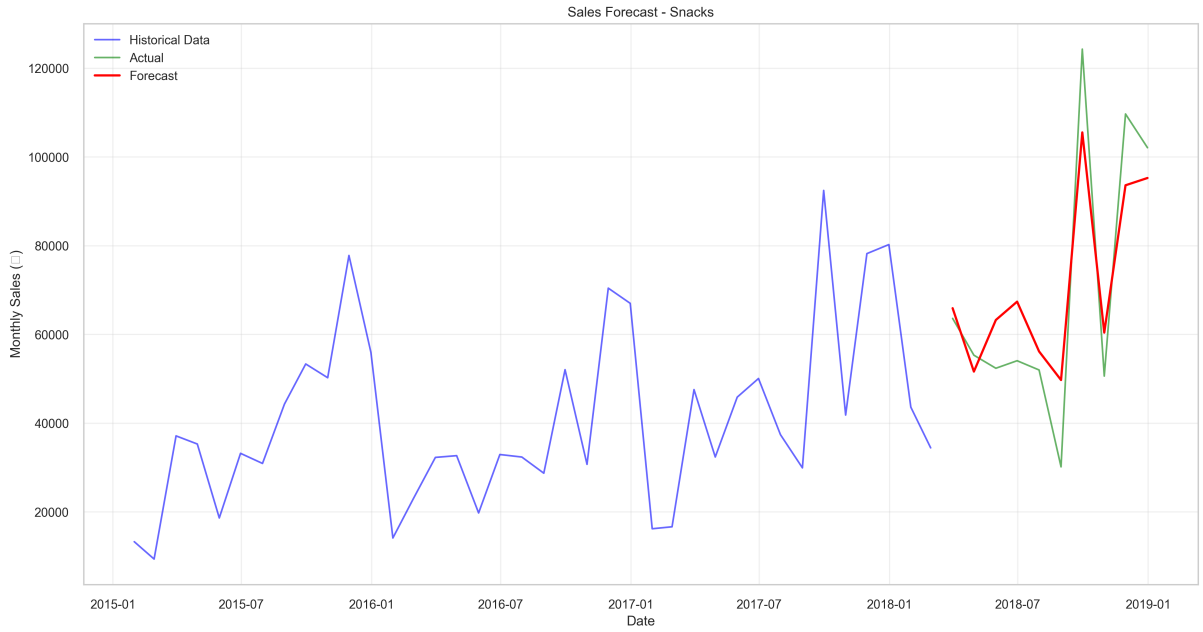

Figure 6: Sales Forecast - Snacks. Shows the best forecasting performance among all categories, with predictions closely matching actual values.

6 Key Insights

1. **Strong Seasonal Patterns**:
    - All categories show significant seasonal variation
    - Monsoon season consistently dominates sales volumes

- Summer shows systematic decrease across categories

2. **Category-Specific Trends**:
   - Staples (Food Grains, Oil & Masala) show stable patterns
   - Fresh categories (Fruits & Veggies) show higher volatility
   - Processed foods (Snacks, Beverages) show moderate seasonality
3. **Forecasting Effectiveness**:
   - Best performance in stable categories (Snacks, Oil & Masala)
   - Higher uncertainty in fresh produce categories
   - Seasonal patterns well-captured across all categories

7 Conclusion

The analysis provides valuable insights for supply chain optimization:

- **Inventory Management**:
  - Increase stock levels before monsoon season
  - Reduce inventory during summer months
  - Category-specific stocking strategies needed
- **Business Planning**:
  - Align promotional activities with seasonal patterns
  - Adjust pricing strategies seasonally
  - Plan resource allocation based on predicted demand
- **Future Improvements**:
  - Incorporate external factors (weather, events)
  - Develop category-specific forecasting models
  - Include price elasticity analysis